

Free Plugin Gives Claude Code Persistent Memory Across Sessions

Quarry learns from every conversation and recalls what you've researched before:
locally, with no cloud accounts or data leaving your machine

Open source plugin captures web research, session transcripts, and project files as
searchable knowledge that compounds over time

Press Release

PORTLAND, OR, September 2026. Today, Punt Labs announced Quarry, a free, open-source tool that gives Claude persistent memory and instant access to everything on your computer. Quarry indexes PDFs, scanned documents, spreadsheets, presentations, web pages, notes, source code, and images (over 20 document formats) into a local semantic search engine, then exposes that knowledge to Claude Code and Claude Desktop through the Model Context Protocol [1]. As a Claude Code plugin, Quarry goes further: every web page Claude researches, every debugging conclusion reached, every conversation transcript is automatically captured before it can be lost to context compaction. Quarry runs entirely on the developer's machine, is open source under the MIT license, and works with the Claude Code plugin system that has grown to over 9,000 extensions (see [FAQ 12](#)).

Problem

Developers who use Claude Code for serious, multi-session work (refactoring systems, debugging production issues, designing architectures) lose everything they learn at the end of every conversation. A developer spends forty-five minutes with Claude tracking down a connection timeout, reads three Stack Overflow answers, identifies the root cause, and ships the fix. Two weeks later, the same class of issue appears in a different service. Claude starts from zero: it searches the web again, reads the same answers, discovers the same root cause. The developer recognizes the pattern but Claude does not.

People who use Claude for research, analysis, or writing hit the same wall. They spend 5–15 minutes gathering context before they can ask their real question. They upload PDFs to Projects. They copy-paste from old documents. They point Claude Code at directories and hope the right files get picked up. When the context window fills, they start over. A developer with ten years of reference PDFs and design documents cannot make any of that knowledge available to Claude without manually curating it first.

The knowledge is generated. The work is done. And then it vanishes (see [FAQ 13](#)).

Solution

Quarry works at two levels: as a search engine for your files, and as a plugin that captures knowledge automatically.

Search engine. Install with `uv tool install punt-quarry` and `quarry install`. Point it at your files: register a directory with `quarry register ~/Documents/research && quarry sync`, or ingest a web page with `quarry ingest https://example.com/report`. From that moment, Claude can search your indexed knowledge by meaning rather than keywords. Every format is handled automatically: text PDFs are extracted, scanned documents and images are OCR'd locally,

spreadsheets are serialized to preserve tabular structure, presentations are split slide-by-slide, HTML pages are stripped of boilerplate, and source code is parsed into semantic units.

Ambient knowledge plugin. As a Claude Code plugin, Quarry captures knowledge passively through hooks. When Claude fetches a web page, a hook queues the URL for background ingestion. Before context compaction, a hook captures the conversation transcript. Your project directory is registered at session start. Every hook is fail-open and non-blocking: you work normally, and the knowledge base grows from the natural flow of every session. Per-project configuration via `.punt-labs/quarry/config.md` lets you disable individual hooks.

Because Quarry runs entirely on your machine, your documents never leave your laptop. No cloud service, no API key, no subscription.

Customer Quote

"I've been using Claude Code full-time for seven months. The thing that frustrated me most wasn't the model; it was the amnesia. I'd debug something, figure out the fix, and three weeks later Claude would research the same problem from scratch while I watched. With Quarry's hooks capturing web pages from that first session, the second time Claude found the answer locally in two seconds instead of spending five minutes searching Stack Overflow. After a month, I had a searchable archive of every piece of research Claude had ever done for me, and I never explicitly saved any of it."

— **Sarah Lindqvist**, Staff Engineer at a Series C startup

Getting Started

No accounts. No API keys. Under five minutes:

1. **Install:** `uv tool install punt-quarry && quarry install` downloads the embedding model, installs the daemon, and configures Claude Code and Claude Desktop automatically.
2. **Ingest:** `quarry register ~/Documents/work && quarry sync` indexes an entire directory. Or ingest a web page directly with `quarry ingest https://example.com/report`.
3. **Work normally.** Open Claude Code or Claude Desktop and ask a question. Claude calls Quarry's search tool automatically. The plugin hooks capture web research and session transcripts in the background.

Spokesperson Quote

"The bottleneck for LLM-assisted work isn't the model; it's the context. People have years of accumulated knowledge on their hard drives, and none of it is accessible to their LLM. We built Quarry to solve two problems: unlocking the knowledge trapped in files, and preserving the knowledge generated during every coding session that vanishes when the conversation ends. The Model Context Protocol made the first possible; the Claude Code hook system made the second possible. Quarry doesn't just index your files; it makes Claude itself smarter."

— **JM Freeman**, Creator of Quarry

Call to Action

Quarry is available now at <https://pypi.org/project/punt-quarry/> and <https://github.com/punt-labs/quarry>. It is free, open source (MIT license), and always will be. Install with `uv tool install punt-quarry`, run `quarry install`, and start searching.

Frequently Asked Questions

External FAQs

Q1: What is Quarry and who is it for?

Quarry is a local semantic search engine and Claude Code plugin that gives your coding sessions persistent memory. It indexes files across 20+ formats, captures web research and conversation transcripts automatically via hooks, and makes all of it searchable by meaning using hybrid search (vector + BM25 fused via RRF). It is for developers who use Claude Code or Claude Desktop for multi-session work and are tired of Claude re-researching topics or manually uploading files before every session.

Q2: How is this different from uploading files to Claude Projects?

Projects require you to select and upload files before every conversation, the context window limits how many you can include, and when you start a new conversation, you start from scratch. Quarry inverts this: index your files once, and Claude searches them on demand during any conversation. You don't decide what's relevant; Claude does. There is no context window limit because Quarry returns only the relevant passages, not entire files.

Q3: How is this different from pointing Claude Code at a folder?

When Claude Code reads a directory, it reads files sequentially, consuming context window with every file it opens. For a folder with 50 documents, Claude Code may exhaust its context before finding the relevant passage. Quarry pre-indexes the content and returns only matching chunks, leaving the context window for reasoning rather than reading.

Q4: How is this different from CLAUDE.md or memory files?

CLAUDE.md is static instructions that you write and maintain manually. It tells Claude *how* to behave. Quarry captures *what Claude learned* (web pages, debugging conclusions, design rationale) and makes it searchable by meaning. You do not write or maintain the knowledge base; it builds itself from every session. CLAUDE.md says "use ruff for linting." Quarry remembers that the last time you debugged a ruff configuration issue, the root cause was a conflicting pyproject.toml section, and the Stack Overflow answer that explained it.

Q5: What file formats does Quarry support?

PDF (both text and scanned/image pages), images (PNG, JPG, TIFF, BMP, WebP), spreadsheets (XLSX, CSV), presentations (PPTX), HTML, text files (Markdown, LaTeX, plain text, DOCX), and source code in 30+ languages parsed into semantic units via Tree-sitter AST. Every format is converted to text optimized for LLM consumption. Spreadsheets and presentation tables are serialized to LaTeX to preserve tabular structure. Source code is chunked by function and class boundaries, not arbitrary line counts. Scanned documents and images are OCR'd locally, with no cloud account needed.

Q6: How does the ambient capture work?

Quarry uses Claude Code's hook system [2], the same event-driven architecture used by plugins like Entire.io for code provenance. Seven hooks ship today (`plugin/hooks/hooks.json`):

- **Session start** auto-registers the project directory so new files are indexed on sync, and injects a knowledge base summary into Claude's context.
- **Post web fetch** auto-ingests every URL Claude fetches into the knowledge base for later semantic search.
- **Post web search** files a scrubbed digest of each web search's results alongside the fetched pages.
- **Post read** (opt-in, off by default) captures prose files read from outside any registered tree, behind an in-tree, secret-path, extension, and size filter.
- **Pre-compact** captures the conversation transcript before context compaction, preserving decisions and discoveries that would otherwise be lost.
- **Session end** captures the full transcript on every close, so a short session that never compacts is still kept.
- **Subagent stop** archives a subagent's own transcript and, when the subagent is a registered ethos identity, distills its final report into that agent's `memory-<handle>` collection (DES-055).

Every hook makes one bounded call to the resident quarryd daemon and returns immediately; it never blocks the conversation. The daemon scrubs, embeds, and indexes in the background. The hooks are fail-open: if Quarry is unavailable, Claude Code works exactly as before. Per-project configuration via `.punt-labs/quarry/config.md` lets you selectively disable individual hooks. `quarry enable <dir>` activates the capture hooks for a project in one command and keeps them scoped: web research and session transcripts route to a separate `<name>-captures` collection so browser tabs and conversation fragments never pollute the file-sync collection you search for code (DES-029). `quarry disable` reverses it. This means a search of your codebase returns code, not the pages Claude read while writing it.

Q7: Does my data leave my computer?

No. Quarry runs entirely on your machine. The vector database (LanceDB), the embedding model (snowflake-arctic-embed-m-v1.5 via ONNX Runtime), the OCR engine (RapidOCR), and all hooks execute locally. No data is sent to any cloud service. Captured web pages are re-fetched and stored locally, not forwarded through a proxy.

One planned feature involves the network by design: an opt-in sync that pushes captured session transcripts to a *user-owned* private git repository for cross-machine continuity. It is off by default and never targets a Punt Labs server. Captures are PII-redacted at write time (secrets, filesystem paths, email addresses, and the local hostname are stripped before a capture file is written; DES-036), so redaction, not trust in a remote, is the mitigation for anything that would leave the machine.

Q8: How much does it cost?

Free. Quarry is open source under the MIT license. There is no paid tier, no usage limits, and no telemetry. The embedding model download is approximately 120 MB (INT8 quantized); after that, everything runs offline.

Q9: Can I use Quarry with tools other than Claude?

Quarry exposes a standard MCP server over stdio and a versioned REST API (/v1, TLS, Bearer token). Any MCP-compatible client can use it. Today that includes Claude Code and Claude Desktop. As other LLM tools adopt MCP, Quarry will work with them without modification. The CLI also works standalone for direct search without any LLM.

Q10: How does search quality compare to keyword search (Spotlight, grep)?

Quarry uses hybrid search: vector similarity for meaning and BM25 full-text for keyword precision, fused via reciprocal rank fusion (RRF). Searching for “authentication logic” finds a function called `verify_credentials` even though the words don’t overlap (vector match), and searching for a specific error code finds the exact line (BM25 match). You get both semantic understanding and keyword exactness in a single query, running locally with a 768-dimension embedding model and Tantivy full-text index. Ranking is not yet perfect: because the dense stage is a bi-encoder, short term-dense chunks (frontmatter, changelog lines) can outrank the passage that best answers a query. Fixing this ranking bias is the active retrieval-quality work tracked in [FAQ 16](#).

Q11: Do I have to manually index everything?

No. The Claude Code plugin captures knowledge passively through its hooks [2] (see [FAQ 6](#) for detail). You work normally; the knowledge base grows in the background. Explicit CLI commands are available for one-off ingestion.

Internal FAQs

Value & Market

Q12: What is the total addressable market?

Claude Code had over 115,000 developers by mid-2025 [3], with Anthropic reporting doubled weekly active users and annualized revenue exceeding \$2.5B by early 2026 [4]. The plugin ecosystem grew from zero to 9,000+ extensions in under five months [5]. The broader AI-assisted coding market (Cursor, GitHub Copilot, Windsurf) totals millions of developers.

The target segment is *multi-session power users*: developers doing complex work spanning days or weeks. The Qodo survey found 41–52% of developers report context-related frustration [6]. A quarter of frustrated Claude Code users is tens of thousands. Because Quarry is free, TAM is measured in adoption. Success means becoming the default way Claude Code sessions persist knowledge.

Q13: What evidence do we have that developers want this?

Four categories of evidence:

Survey data: The Qodo “State of AI Code Quality” survey found that context-related frustration affects 41–52% of developers (increasing with seniority), and “improved contextual understanding” is the single most-requested AI improvement at 26% of votes. The Stack Overflow 2025 survey (49,000+ respondents) found 81% have security/privacy concerns about AI data use [7]; local-first architecture directly addresses this.

Market signal: SageOx [8] positions itself as “context infrastructure for human-agent collaboration.” Entire.io, founded by former GitHub CEO Thomas Dohmke with \$60M in seed funding [9], captures the reasoning behind AI-generated code via a git shadow branch [10]. Both companies are building for the same problem.

Behavioral: Claude Code’s own memory features (CLAUDE.md, auto-memory) are static and manually maintained; they capture instructions, not learned knowledge. The gap between “remember my preferences” and “remember what we researched” is the gap Quarry fills.

Adjacent: Rewind/Limitless raised \$33M+, was acquired by Meta in December 2025, and shut down both its Pendant hardware and Rewind desktop app [11, 12]. The demand was real; the cloud-dependent model was fragile. Quarry’s local-first architecture is resilient to this failure mode.

Q14: Who are the competitors and why will we win?

The competitive field splits into six categories; no single tool covers Quarry’s niche. See the Competitive Landscape appendix for the full analysis.

MCP-native code search (DeepContext, claude-context, mcp-local-rag, Chroma MCP): Code-only, typically require cloud API keys, no agent memory. Quarry indexes 20+ formats, runs fully offline, and provides agent memory with identity tagging and temporal decay.

Documentation servers (Context7): Pre-indexed public library docs. Complementary, not competitive. Quarry indexes *your* files; Context7 indexes *everyone’s* libraries.

Personal knowledge bases (Khoj, Obsidian plugins, Mem.ai): GUI-first, human-interactive, no MCP server, no agent memory concept.

Session capture (Entire.io, SageOx, Screenpipe): Record what happened. Quarry adds semantic search, making captured knowledge retrievable by meaning across sessions.

RAG frameworks (RAGFlow, AnythingLLM, LlamaIndex): Toolkits requiring assembly. Quarry is opinionated and zero-configuration.

IDE built-in indexing (Cursor, Windsurf): Codebase search locked inside their editors, cloud-dependent, code-only, invisible to external agents.

Platform risk: Anthropic building ambient memory natively into Claude Code is the most dangerous response. Quarry's residual value: (1) cross-tool portability via MCP, CLI, and HTTP API; (2) user-owned data in an open format (LanceDB); (3) 20+ format indexing; (4) open source and auditable. Secondary risks: Cursor or Windsurf exposing indexes via MCP; Khoj shipping an MCP server; a vector DB vendor shipping a batteries-included search product.

Q15: What is the user acquisition strategy?

Quarry has no installed base yet. Acquisition depends on three channels:

PyPI: Users searching for "claude mcp," "local RAG," or "semantic search" find Quarry on PyPI. The one-line install (`curl ... | sh`) and zero-account setup reduce friction.

Claude Code plugin marketplace: The marketplace is the primary discovery surface for Claude Code users. Quarry is listed and installable with one command.

GitHub and community: Engineering quality (a comprehensive automated test suite, strict typing, clean architecture) attracts contributors. Presence in Claude community forums and word-of-mouth among power users.

The risk is obscurity (see [FAQ 25](#)). No channel has proven traction yet.

Technical

Q16: What are the major technical risks?

Embedding model quality (medium risk): snowflake-arctic-embed-m-v1.5 (768 dimensions, 512 token context) performs well for general text but may underperform on domain-specific content (medical, legal, financial jargon). Mitigation: the backend abstraction allows swapping models without code changes: a model upgrade is a configuration change, not an architecture change.

Retrieval ranking quality (medium risk): Quarry ranks per-chunk with a bi-encoder (mean-pooled dense embedding) fused with BM25 via RRF. A bi-encoder rewards *token density*, not relevance: short, term-dense chunks (document frontmatter, changelog lines, tables of contents) can systematically outrank the substantive passages that actually answer a query. On the live index, the query "Predicate Logic in Z" returns a manual's YAML frontmatter at #1 and a one-line CHANGELOG entry at #2, with the passage that answers the question ranked #3. This is distinct from the already-fixed FTS fake-1.00 scoring bug; here every score is proper bounded cosine and the ranker is simply under-valuing substance. The mitigation is at the embedding layer, not a reranker: context-aware embeddings (contextual embeddings, or late chunking) plus cheap metadata down-weighting, keeping RRF at $k = 60$ as the validated, calibration-free fusion floor. An off-the-shelf cross-encoder reranker degrades quality on technical corpora, so one is adopted only if it wins on Quarry's own corpus. This direction touches DES-004's "the embedding model is fixed" stance: late chunking needs a long-context model and may require swapping off arctic-embed's 512-token cap, an open decision the evaluation harness is now in place to settle. Phase 0/1 have landed on main (make eval, measuring per-bucket MRR/success@k plus a metadata-pollution diagnostic on Quarry's own data); full-fixture curation and the lever bake-offs are the remaining work (docs/retrieval-quality-improvements.md, docs/eval-harness-design.md).

OCR accuracy on degraded scans (low risk): RapidOCR produces good-enough text for semantic search but lower character accuracy than commercial OCR. Mitigation: semantic search is tolerant of OCR errors because it matches by meaning, not exact characters.

LanceDB scalability (low risk): At tens of thousands of documents, query latency and index size may become concerns. Mitigation: the database abstraction allows migration to a different backend, and named databases allow partitioning by project.

Hook latency (low risk): All hooks must return fast to avoid blocking Claude Code. Each hook makes a single bounded loopback POST to the resident daemon and returns; there is no embedding and no engine import in the hook path. The SubagentStop hook, which Claude Code treats as blocking, returns an empty result on every path. The dispatcher (quarry-hook) is stdlib-only at import and lazily loads just the thin client.

Q17: What dependencies exist on external systems?

MCP protocol: Quarry depends on the MCP specification remaining stable and Claude clients continuing to support MCP tool servers. Risk is low: Anthropic is investing heavily in MCP, and multiple third-party clients now support it.

Claude Code hook system: The ambient capture layer depends on Claude Code's hook events (SessionStart, PostToolUse, PreCompact, SessionEnd, SubagentStop) remaining stable. Risk is low: the hook system is actively developed and used by multiple shipping plugins [2].

Embedding model hosting: The ONNX model is downloaded from Hugging Face Hub at install time. If removed, existing installations continue to work (model cached locally). Mitigation: model is pinned to a specific git revision.

Tree-sitter grammars: Mature, widely-used open-source projects with low abandonment risk. Quarry operates fully offline after installation.

Q18: What is the development timeline and current status?

The press release above is written Working-Backwards: an aspirational future framing of what adoption looks like. The code, by contrast, is real and shipping today: Quarry is live on PyPI. The gap this document tracks is adoption, not availability (see [FAQ 26](#)). Current capabilities:

Shipped: PDF ingestion (text + local OCR), image OCR, spreadsheet ingestion (XLSX/CSV), presentation ingestion (PPTX), HTML parsing, URL and sitemap crawling, text file parsing, DOCX, source code parsing (30+ languages via Tree-sitter), semantic search with metadata filtering, hybrid search (vector similarity + BM25 full-text fused via reciprocal rank fusion), agent memory with identity tagging and temporal decay (via ethos integration) plus its write loop (a closed, server-validated `memory_type` vocabulary; `quarry learn` lessons with retrieval preference, DES-053; `quarry missions sync` filing evaluator feedback into the worker's memory; and SubagentStop distilling a subagent's final report into its own memory, DES-055), directory sync with `.gitignore/.quarryignore` support and an always-on filesystem watch that re-indexes on change (DES-045), named databases and collections, MCP server with 13 tools, CLI with `-version` and plain-text help, a versioned `/v1` REST API with Bearer token authentication (REST only; the in-daemon MCP WebSocket route was removed under DES-031 v2.2), Claude Code plugin with seven hooks (session start auto-registration plus knowledge-base briefing; post web fetch, post web search, and opt-in post read capture; pre-compact and session-end transcript preservation; subagent-stop capture and distillation), the `quarryd` daemon (`launchd/systemd`) with `quarry mcp` as a thin stdio client of it, ONNX provider auto-detection with CUDA/GPU acceleration (`curl|sh` installer auto-installs `onnxruntime-gpu` on NVIDIA machines; uses FP16 model), researcher subagent for combined local + web search, CLAUDE.md injection for agent

discovery, ethos extension for per-identity session context; remote server support with TLS everywhere and TOFU certificate pinning (DES-020, DES-021, DES-022); quarry enable/disable unified project activation with scoped <name>-captures collections (DES-029); the SessionStart knowledge-base briefing (DES-013); bounded progressive sync with mid-flush visibility and within-file crash-resume (DES-034); the retrieval-quality eval harness (make eval, an internal dev/CI tool reporting per-bucket MRR/success@k plus a metadata-pollution diagnostic, built on a parameterizable retrieval layer (src/quarry/retrieval/) shared by production and eval; Phase 0/1); and capture-path write-time PII redaction that scrubs secrets, PII, and profanity from session transcripts and WebFetch captures before they are written (DES-036). A comprehensive automated test suite backs all of it, with strict type checking (mypy + pyright).

Roadmap: index-wide PII detection over arbitrary ingested files, a Google Drive connector, the quarryd host-adaptor contract for non-Claude agent hosts (requirements drafted, not yet designed), and the retrieval-ranking lever bake-offs (context-aware embeddings and metadata down-weighting) that the eval tooling is built to settle (see [FAQ 16](#)).

Q19: What is the scaling story?

Quarry is designed for personal-scale workloads: thousands to tens of thousands of documents on a single machine.

On an M2 MacBook Air, the ONNX embedding model sustains ~11 chunks/second. A 10,000-file directory (~70,000 chunks) takes roughly 1.5–2 hours, and results are queryable *during* the sync, not only after it finishes. Bounded progressive commit (DES-034) flushes chunks to LanceDB in fixed-size windows: peak memory is bounded regardless of file or collection size, each flush publishes a new MVCC version that concurrent search sees immediately, and a crash resumes *within* the file it was embedding rather than restarting the collection. Subsequent syncs are incremental.

Query latency is constant regardless of database size because LanceDB uses IVF-PQ indexing. Quarry supports multi-user access to a shared LAN daemon (e.g. a team sharing a GPU machine), but does not target multi-tenant deployments with per-user isolation. The Fly.io deployment that serves the [punt-labs.com](#) chat widget's RAG backend is a single-tenant instance.

Q20: What happens as the knowledge base grows?

LanceDB is designed for datasets far larger than a personal knowledge base. Search latency remains sub-second up to millions of vectors; 10,000–50,000 documents (years of heavy use) is well within range. Disk usage scales at approximately 1–5 MB per 100 documents for embeddings and metadata. Captured content (web pages, transcripts) adds more but is bounded by dedup logic: the same URL is not re-ingested.

quarry delete lets users prune old or low-value collections.

Business

Q21: What is the revenue model?

None. Value is measured in adoption and community contribution, not revenue.

Q22: What are the key metrics for success?

- **PyPI installs:** Monthly download count as a proxy for adoption. Baseline as of March 2026: ~765 monthly downloads. Tiered thresholds at 6 months: below 500/month triggers a go/no-go decision on continued development; 1,000/month indicates baseline adoption; 5,000/month indicates product-market fit.
- **GitHub stars:** Community interest signal. Target: 500 stars within 6 months; 2,000 indicates breakout.
- **Active MCP connections:** Users with Quarry configured in Claude Code or Desktop. Not directly measurable without telemetry; inferable from support requests and community engagement.
- **Contributor count:** External contributors submitting PRs. Target: 5+ contributors within the first year.
- **Format coverage:** PDF, images, text, code, XLSX, PPTX, HTML are all shipped. Remaining targets: EPUB, email (EML/MBOX), audio transcription.

Q23: Why now? What has changed?

Three shifts converged:

MCP maturity: The Model Context Protocol shipped in late 2024 and is now supported by Claude Code and Claude Desktop. A local tool can be a native part of an LLM's reasoning process, not an external API the user invokes manually.

Claude Code plugin ecosystem: The hook system (PostToolUse, PreToolUse, PreCompact, SessionStart) is stable and used by shipping plugins [2]. The ecosystem grew to 9,000+ plugins in under five months [5]. Quarry applies this infrastructure to knowledge persistence.

Local inference viability: ONNX Runtime, quantized embedding models, and LanceDB make it feasible to run a complete semantic search pipeline on a laptop CPU in under a second per query. GPU acceleration is now zero-config: the installer auto-detects NVIDIA hardware and swaps in `onnxruntime-gpu` with FP16 models, giving the same tool a hardware fast path without user configuration.

Q24: What are we not building?

- **A cloud service.** Quarry will never require an account, subscription, or data upload. Local-first is a permanent design constraint, not a temporary limitation.
- **A multi-tenant platform.** Quarry supports multi-user access to a shared daemon on the local network (e.g. a team sharing a GPU machine), authenticated via Bearer token. But there is no per-user isolation, no access controls, no permissions, and no separate views of the data. Everyone sees the same knowledge base.
- **Media search.** Quarry does not find images by visual content or match audio. It extracts *text* from every format and indexes the knowledge inside.
- **A note-taking app.** Quarry indexes existing files. It does not provide an editor, a tagging system, or a filing structure.
- **A general-purpose RAG framework.** No pipeline assembly, no pluggable retrievers, no chain-of-thought orchestration. Quarry is deliberately opinionated and zero-configuration.
- **Structured knowledge extraction.** Quarry indexes raw content and lets semantic search surface what is relevant. It does not extract summaries, decisions, or intent from captured content.

- **Git write-side provenance.** Quarry does not add metadata to git commits: no checkpoint trailers, shadow branches, or commit annotations. It *reads* from git (commits made during a session) to enrich captured transcripts with artifact context.

Q25: What does failure look like?

Three plausible failure scenarios:

Obscurity: Quarry ships, works well, and nobody discovers it. After 6 months, monthly PyPI installs plateau below 200. *Response:* The most likely failure mode. Mitigation: active presence in Claude community forums. If installs plateau, invest in content marketing before building more features.

First-party displacement: Anthropic ships native file indexing or session memory in Claude Code. *Response:* That validates the thesis. Quarry still serves users who need local-only indexing, offline access, or formats Anthropic doesn't cover.

Capture noise exceeds signal. The knowledge base fills with low-value web pages and irrelevant transcript fragments, degrading search quality. *Response:* Start conservative (web research + compaction only). If signal-to-noise is poor, add quality filtering: URL pattern exclusions, minimum page length, dedup by content similarity. Two structural mitigations already ship. Captures are filed into scoped <name>-captures collections that the code-search collection never sees (DES-029). Agent memory is written through a closed, server-validated `memory_type` vocabulary as typed, attributed documents (mission rounds, subagent reports) rather than raw transcript dumps (DES-055), so a typed `find` filter returns memories, not noise.

Q26: What is the next step to validate adoption?

The riskiest assumption is no longer technical. The capture hooks, the SessionStart knowledge-base briefing, remote sharing, and progressive sync all ship and work for the author. What remains unproven is *value to others*: whether a developer who is not the author installs Quarry, indexes enough of their own material, and comes back because it saved them a redundant round of research. Until that happens the product is a well-built hypothesis, not a validated one, which is why this document stays at hypothesis stage.

The next step is to put Quarry in front of a small cohort of external Claude Code power users, via the plugin marketplace and Claude community forums, and measure whether they reach the moment where Claude surfaces a document they had forgotten they owned. Retention past the first week, not raw install count, is the signal. If early adopters index and return, the thesis holds; if they install and drift away, the constraint is discovery-of-value, and the response is onboarding and content, not more features.

Risk Assessment

Risk	Rating	Assessment
Value	Medium	Multiple startups and survey data validate the problem [6, 9]. Discovery and activation are the primary risks: demand is proven, but users must find Quarry and index enough content to experience the value.
Usability	Medium	For CLI-native users, three commands to value. Auto-configures MCP. No accounts, no API keys. The SessionStart knowledge-base briefing ships and surfaces the index to Claude automatically. The residual risk is the first-value moment: a user must index enough of their own material before Claude can surface something they had forgotten, and that activation step is unproven with external users.
Feasibility	Low	The product is live on PyPI with a comprehensive automated test suite and strict type checking. Core pipeline, daemon architecture, plugin hooks, MCP server, hybrid search, agent memory, CUDA acceleration, TLS remote server support, quarry enable/disable project activation, and bounded progressive sync are all shipped. The one open technical question is retrieval ranking quality (see FAQ 16); remaining roadmap items are additive features, not architectural risks.
Viability	Low	No revenue model to validate. Zero marginal cost (runs on user hardware). MIT license.

Value risk is the dominant concern. The thesis: local semantic search combined with passive knowledge capture changes how developers work with AI coding tools: your LLM has access to everything you know, and knowledge generated during sessions compounds instead of evaporating. Discovery is the immediate challenge: users must find Quarry and invest enough initial indexing to experience Claude finding a document they forgot they had. The recall briefing that surfaces local knowledge to Claude already ships; what remains unproven is whether users beyond the author adopt Quarry and return once they have. That is the value-to-others question this document stays at hypothesis stage to answer (see [FAQ 26](#)).

Feature Appendix

Defines scope. Features are categorized to prevent scope creep.

Core (Shipped)

- F1. Local semantic search** — Index local files and retrieve passages by meaning across 20+ document formats.
- F2. MCP server** — 13 tools covering search, ingestion (`ingest`, `remember`, `learn`), management, database operations, and `missions_sync`. Exposes Quarry to Claude Code and Claude Desktop as a native tool.
- F3. Directory registration and sync** — Bulk ingestion workflow: register a directory once, sync incrementally as files change. Respects `.gitignore` and `.quarryignore`.
- F4. Local OCR** — Extract text from scanned documents and images on-device via RapidOCR with no cloud account required.
- F5. Named databases** — Separate knowledge bases by project or domain.
- F6. One-command install** — `quarry install` auto-configures MCP for Claude Code and Claude Desktop, downloads `mcp-proxy`, and installs the daemon.
- F7. Claude Code plugin** — Plugin with seven hooks: auto-register the project directory on session start; capture what Claude fetches (post web fetch), searches (post web search), and, opt-in, reads outside the tree (post read); preserve the transcript before context compaction (pre-compact) and on every close (session end); and archive each subagent's transcript, distilling a registered identity's final report into its own memory (subagent stop). All hooks fail-open, per-project configurable, and thin daemon clients.
- F8. Daemon architecture** — Single background process (`quarryd`, supervised by `launchd`/`systemd`) holds the engine (embedding model, LanceDB, pipeline) and serves a versioned `/v1` REST API over TLS. The CLI, the `quarry mcp stdio` server, the hooks, and the library are pure clients of it; no client process loads the model (DES-031 v2.2).
- F9. Always-on watch/index loop** — `quarryd` watches every registered directory (`FSEvents`/`inotify`, debounced, with a periodic safety sweep) and re-indexes on change, so a registered tree stays current with no human action; `quarry sync` is a one-shot pass on top (DES-045).
- F10. Researcher agent** — Combines local Quarry search with web research. Searches locally first, fills gaps from the web, and auto-ingests findings so research compounds across sessions.
- F11. Hybrid search** — Vector similarity + BM25 full-text fused via RRF. Semantic understanding plus keyword precision.
- F12. Agent memory** — Per-agent memory with identity tagging via ethos integration. A closed, server-validated `memory_type` vocabulary (`fact`, `observation`, `opinion`, `procedure`; `lesson` reserved for `learn`; anything else is a 400) with temporal decay for the agent-owned types and a retrieval boost for lessons. The write loop fills the shelf without a model in the loop: a versioned memory guide in every identity's session context, `quarry missions sync` filing each frozen ethos mission round's evaluator feedback into `memory-<worker>`, and `SubagentStop` distilling a subagent's final report into `memory-<handle>` (DES-055). Agents remember across sessions on the same machine; cross-machine continuity requires a remote daemon.
- F13. CUDA/GPU acceleration** — Auto-detects NVIDIA hardware and uses FP16 model via `CUDAExecutionProvider`. Installer swaps `onnxruntime` for `onnxruntime-gpu` with rollback on failure.

- F14. HTTP API with authentication** — Bearer token on all endpoints except /health. Server refuses non-loopback binding without an API key. Enables secure LAN access to a GPU daemon.
- F15. Ethos integration** — Injects per-identity agent personality, memory instructions, and team context into sessions automatically.
- F16. Remote server support** — `quarry login/logout/remote` connects to a remote daemon (e.g. a LAN GPU machine): credentials in the OS keyring, transparent MCP reconfiguration, sharing a single GPU-accelerated instance. TLS everywhere with TOFU certificate pinning secures every connection including localhost; all data commands route to the daemon with no split horizon; a single `-network installer` path sets it up (DES-020, DES-021, DES-022).
- F17. Knowledge recall briefing** — `SessionStart` injects a knowledge-base summary into Claude's context and nudges it to check local knowledge before searching the web, so the model knows what it already has (DES-013).
- F18. Project activation** — `quarry enable/disable` sets up file sync, passive captures, and agent memory for a project in one command, routing web research and transcripts to a scoped `<name>-captures` collection so they never pollute the file-sync collection (DES-029).

Should Do

- F19. Retrieval-quality eval harness** — A `make eval` tool (internal dev/CI tooling, not an end-user command) over a committed query set, reporting per-bucket MRR and `success@k` plus a metadata-pollution diagnostic. Phase 0 (the parameterizable retrieval layer in `src/quarry/retrieval/`) and Phase 1 (`make eval` with `ranx` metrics and a known-item baseline) have landed on main; full-fixture curation and the lever bake-offs (metadata down-weighting and context-aware embeddings, whether contextual embeddings or late chunking) remain, with graded relevance judgments and `nDCG@10` deferred to Phase 2 (see [FAQ 16](#)).
- F20. Google Drive connector** — Ingest cloud-hosted documents without manual download.
- F21. Index-wide PII detection** — General PII detection over arbitrary ingested files, before content enters the search index. Distinct from the capture-path write-time redaction that has already landed on main (session transcripts and `WebFetch` captures; DES-036): this extends redaction to deliberately-ingested documents.

Won't Do

- F22. Multi-tenant platform** — Multi-user LAN access is supported (shared daemon, Bearer token auth). Multi-tenancy is not: no per-user isolation, no access controls, no permissions.
- F23. Media search by visual or audio content** — Quarry extracts text from every format. It does not match images by visual similarity or search audio.
- F24. Note-taking or filing system** — Indexes existing files. No editor, tagging, or organizational structure.
- F25. General-purpose RAG framework** — Deliberately opinionated and zero-configuration. No pipeline assembly, no pluggable retrievers.
- F26. Hosted web UI** — Team browsing of shared knowledge is outside scope.
- F27. Structured knowledge extraction** — Extraction is lossy and adds complexity. Semantic search over raw content is sufficient.
- F28. Git write-side provenance** — No checkpoint trailers, shadow branches, or commit annotations. Quarry reads from git to enrich transcripts; it does not write to git.

Competitive Landscape

Surveyed March 2026. The field divides into six categories; no single tool covers Quarry's niche.

MCP-Native Code Search

Tools that index code and serve results to AI coding agents via MCP.

Tool	Differentiator	Weakness vs Quarry
DeepContext	AST-level symbol awareness, Jina reranking	Code-only (TS/Python). Cloud API key required. No agent memory.
claude-context (Zilliz)	VSCode extension, AST chunking	Two cloud API keys (Zilliz + OpenAI). Code-only. Not local-first.
mcp-local-rag	Zero setup, fully private	Early-stage. Limited formats. No hybrid RRF. No agent memory.
Chroma MCP	Mature vector DB ecosystem	BYO everything: no parsing, no chunking, no agent memory. A DB, not a product.

Documentation Servers

Pre-indexed public library documentation for AI coding tools.

Tool	Differentiator	Weakness vs Quarry
Context7 (Upstash)	9,000+ pre-indexed libraries. "use context7" trigger.	Public docs only. Cannot index local files. Cloud-only. Different problem.

Assessment: Complementary, not competitive. Context7 solves stale training data for public APIs; Quarry solves local knowledge retrieval. A "use quarry" trigger pattern is worth adopting (see beads backlog).

Personal Knowledge Bases

AI-powered tools for indexing and searching personal documents.

Tool	Differentiator	Weakness vs Quarry
Khoj	Full AI assistant: chat, agents, automations. Multi-platform.	Heavy (Docker + PostgreSQL). No MCP. No CLI-first. Not for coding agents.
Mem.ai	Zero-effort AI auto-organization. Polished consumer UX.	Cloud-only. No MCP. Closed source. No developer focus.
Obsidian + Smart Connections	786K+ downloads. Deep vault integration.	Obsidian-only. Markdown only. No MCP. No document format diversity.
AnythingLLM	Full AI workspace. Built-in LanceDB. GUI-first.	No MCP. Smaller embedding (384-dim). Broader scope dilutes search quality.

Assessment: These tools are GUI-first and human-interactive. Quarry is CLI/MCP-first, designed for AI agents as the primary consumer. The risk is Khoj shipping an MCP server; they have the breadth to compete directly.

Session Capture & Activity Recording

Tools that record developer sessions or screen activity.

Tool	Differentiator	Weakness vs Quarry
Entire.io	Git shadow branch for session metadata. \$60M seed.	Records what happened; no semantic search. Different abstraction.
SageOx	Hosted context platform for enterprise teams.	Enterprise SaaS. Quarry serves individuals with no accounts.
Screenpipe	24/7 screen + audio capture. 16K+ GitHub stars.	Activity capture, not document search. No semantic search. No MCP.
Limitless (ex-Rewind)	Meeting capture with hardware pendant.	Acquired by Meta Dec 2025. Mac app shut down. Cloud-only.

RAG Frameworks

Pipeline toolkits that require assembly to build a search product.

Tool	Differentiator	Weakness vs Quarry
RAGFlow	Enterprise-grade: visual agent builder, team collab.	Heavy (Docker + Elasticsearch). No MCP. No CLI-first UX.
LlamaIndex RAG CLI	Massive ecosystem. Composable.	Framework, not product. Typically needs OpenAI API. No MCP server.

IDE Built-in Indexing

Codebase search locked inside specific editors.

Tool	Differentiator	Weakness vs Quarry
Cursor	Deep IDE integration. @Codebase, @Docs. Shared team indexes.	Locked ecosystem. Cloud embeddings (Turbopuffer). Code-only. No MCP.
Windsurf	Cascade auto-indexes. Remote indexing for 1M+ line repos.	IDE-only. Not accessible to external agents. No document search.

Assessment: The risk is Cursor or Windsurf exposing their codebase indexes via MCP. If they do, Quarry still differentiates on document format diversity, local-first architecture, and agent memory.

Positioning Summary

No existing tool replicates Quarry's combination: zero-API-key local ONNX embeddings, 20+ document formats, hybrid search (vector + BM25 via RRF), agent memory with temporal decay, MCP + CLI surfaces, CUDA acceleration, and daemon architecture for concurrent agent sessions. The three competitive risks worth monitoring:

1. Cursor/Windsurf adding MCP-accessible document search

2. Khoj or a similar knowledge base shipping an MCP server
3. A vector DB vendor (Chroma, LanceDB) shipping a batteries-included search product on top of their database layer

References

- [1] AI Tool Analysis. *Claude Code Plugins Review 2026: 9,000+ Extensions*. Reports 9,000+ Claude Code plugins as of early 2026. 2026. URL: <https://aitoolanalysis.com/claude-code-plugins/>.
- [2] Anthropic. *Claude Code Hooks Reference*. 2026. URL: <https://code.claude.com/docs/en/hooks> (visited on 02/24/2026).
- [3] ppc.land. *Claude Code reaches 115,000 developers, processes 195 million lines weekly*. Reports figures disclosed by Deedy Das (Menlo Ventures) on July 6 2025 — NOT an official Anthropic release. 115,000 developers, 195M lines per week. Anthropic confirmed only “5.5x revenue increase” for July 2025. Use Anthropic Series G announcement for confirmed figures. 2025. URL: <https://ppc.land/claude-code-reaches-115-000-developers-processes-195-million-lines-weekly/>.
- [4] Anthropic. *Anthropic raises \$30 billion in Series G funding at \$380 billion post-money valuation*. Primary Anthropic source: Claude Code WAU doubled since January 1 2026; annualized revenue exceeded \$2.5B; total Anthropic ARR \$14B. Announced February 12 2026. 2026. URL: <https://www.anthropic.com/news/anthropic-raises-30-billion-series-g-funding-380-billion-post-money-valuation>.
- [5] StartupHub AI. *Anthropic’s Claude Code plugins open the floodgates*. Claude Code plugins launched public beta October 9 2025. Ecosystem grew to 9,000+ plugins in under five months. 2025. URL: <https://www.startuphub.ai/ai-news/ai-research/2025/anthropics-claude-code-plugins-open-the-floodgates/>.
- [6] Qodo. *State of AI Code Quality in 2025*. Developer survey: context pain affects 41–52% of developers by seniority; “improved contextual understanding” is the top-requested AI improvement (26% of votes); persistent context reduces frustration from 54% to 16%. Qodo, 2025. URL: <https://www.qodo.ai/reports/state-of-ai-code-quality/>.
- [7] Stack Overflow. *2025 Stack Overflow Developer Survey*. 49,000+ respondents from 177 countries. 84% use AI tools; 46% distrust accuracy (up from 31%); 66% cite “almost right but not quite” as top frustration; 81% have security/privacy concerns about AI data use. Stack Overflow, 2025. URL: <https://survey.stackoverflow.co/2025/>.
- [8] SageOx. *Context Infrastructure for Human-Agent Collaboration*. 2026. URL: <https://sageox.ai> (visited on 02/28/2026).
- [9] TechCrunch. *Former GitHub CEO raises record \$60M dev tool seed round at \$300M valuation*. Confirms Thomas Dohmke (GitHub CEO 2021–2025) founded Entire; \$60M seed at \$300M valuation led by Felicis; largest-ever developer tools seed round. 2026. URL: <https://techcrunch.com/2026/02/10/former-github-ceo-raises-record-60m-dev-tool-seed-round-at-300m-valuation/>.
- [10] Entire. *entireio/cli: Entire is a new developer platform that hooks into your git workflow*. Open-source Checkpoints CLI. Captures prompts, transcripts, tool calls, token usage. Stores on entire/checkpoints/v1 git shadow branch. Supports Claude Code, Gemini CLI, OpenCode, Cursor. 2026. URL: <https://github.com/entireio/cli>.
- [11] TechCrunch. *Meta acquires AI device startup Limitless*. Meta acquired Limitless (formerly Rewind) December 5 2025. Founded by Dan Siroker and Brett Bejcek. Raised \$33M+ from Sam Altman, First Round, a16z, NEA. Team joins Meta Reality Labs. 2025. URL: <https://techcrunch.com/2025/12/05/meta-acquires-ai-device-startup-limitless/>.

- [12] 9to5Mac. *Rewind Mac app shutting down following Meta acquisition*. Confirms Rewind Mac desktop app and Pendant hardware shutdown effective December 2025. 2025. URL: <https://9to5mac.com/2025/12/05/rewind-limitless-meta-acquisition/>.